

Sumário

Arquitetura — Tech Challenge Fase 3	1
1. Desenho da arquitetura	1
2. Descrição do fluxo principal de funcionamento	4
3. Pontos de resiliência	5

Arquitetura — Tech Challenge Fase 3

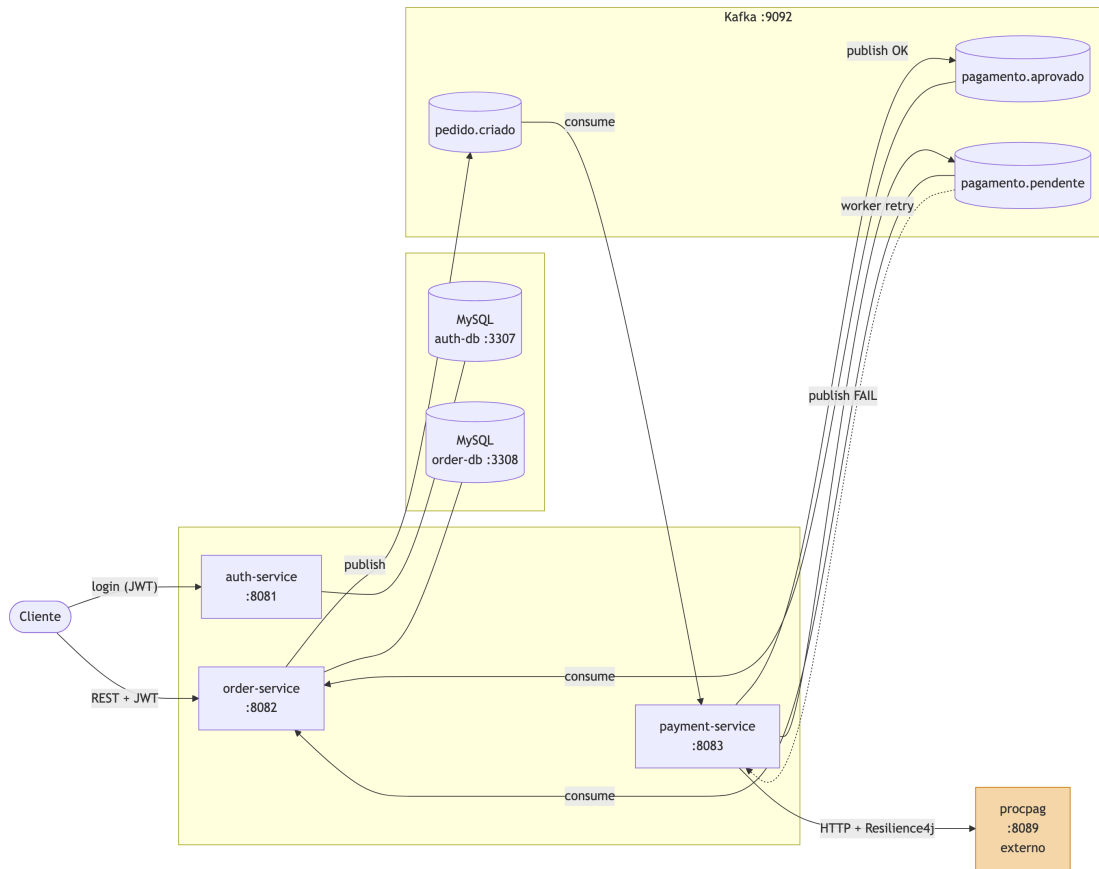
Documento de arquitetura cobrindo os três artefatos pedidos no item 3 do enunciado:

1. Desenho da arquitetura (diagrama de componentes + diagramas de sequência)
2. Descrição do fluxo principal de funcionamento
3. Identificação clara dos pontos de resiliência

1. Desenho da arquitetura

1.1 Diagrama de componentes

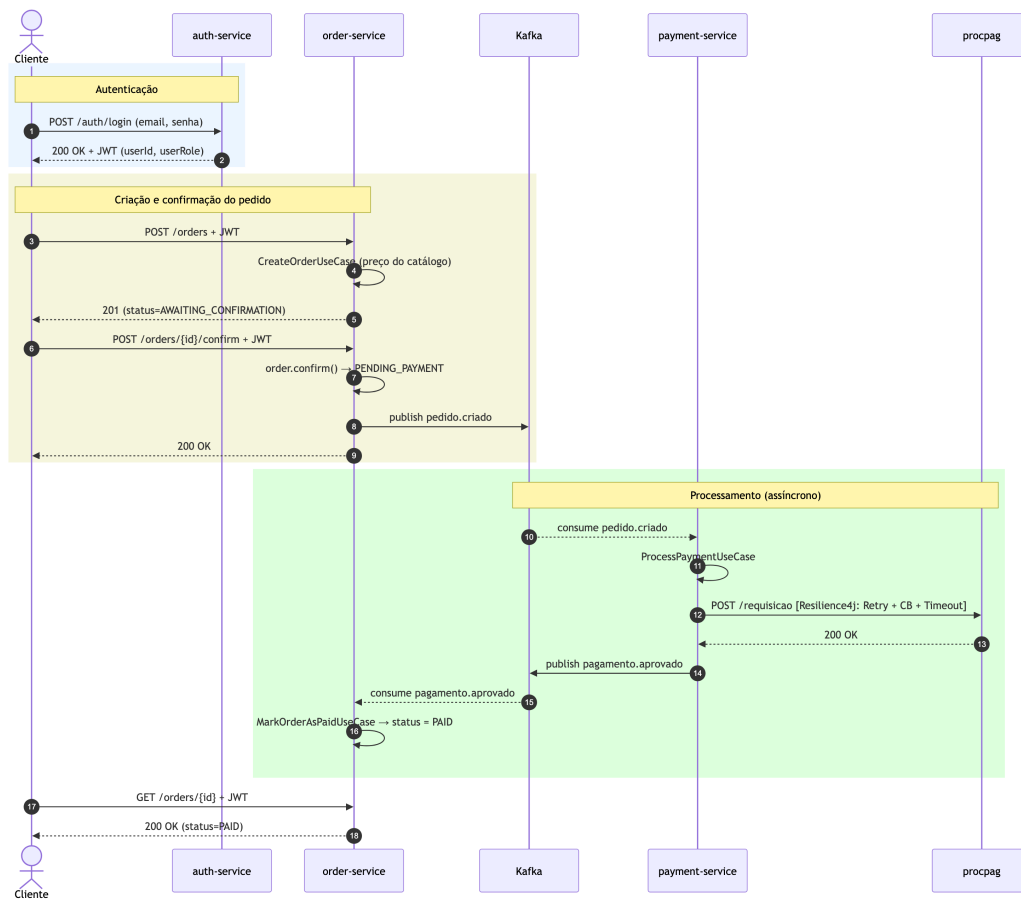
Visão geral dos serviços, dependências e tópicos de mensageria.



Componente	Tecnologia	Responsabilidade
auth-service	Spring Boot 3.5 + Spring Security + JWT	Cadastro e autenticação de clientes
order-service	Spring Boot 3.5 + Spring Kafka + Spring Data JPA	Ciclo de vida do pedido
payment-service	Spring Boot 3.5 + Resilience4j + Spring Kafka	Integração com procpag e publicação do resultado
procpag	Imagem externa fornecida	Serviço de pagamento "eventualmente disponível"
Kafka + ZooKeeper	Confluent 7.6.1	Mensageria assíncrona
MySQL x2	MySQL 8	Persistência isolada por serviço

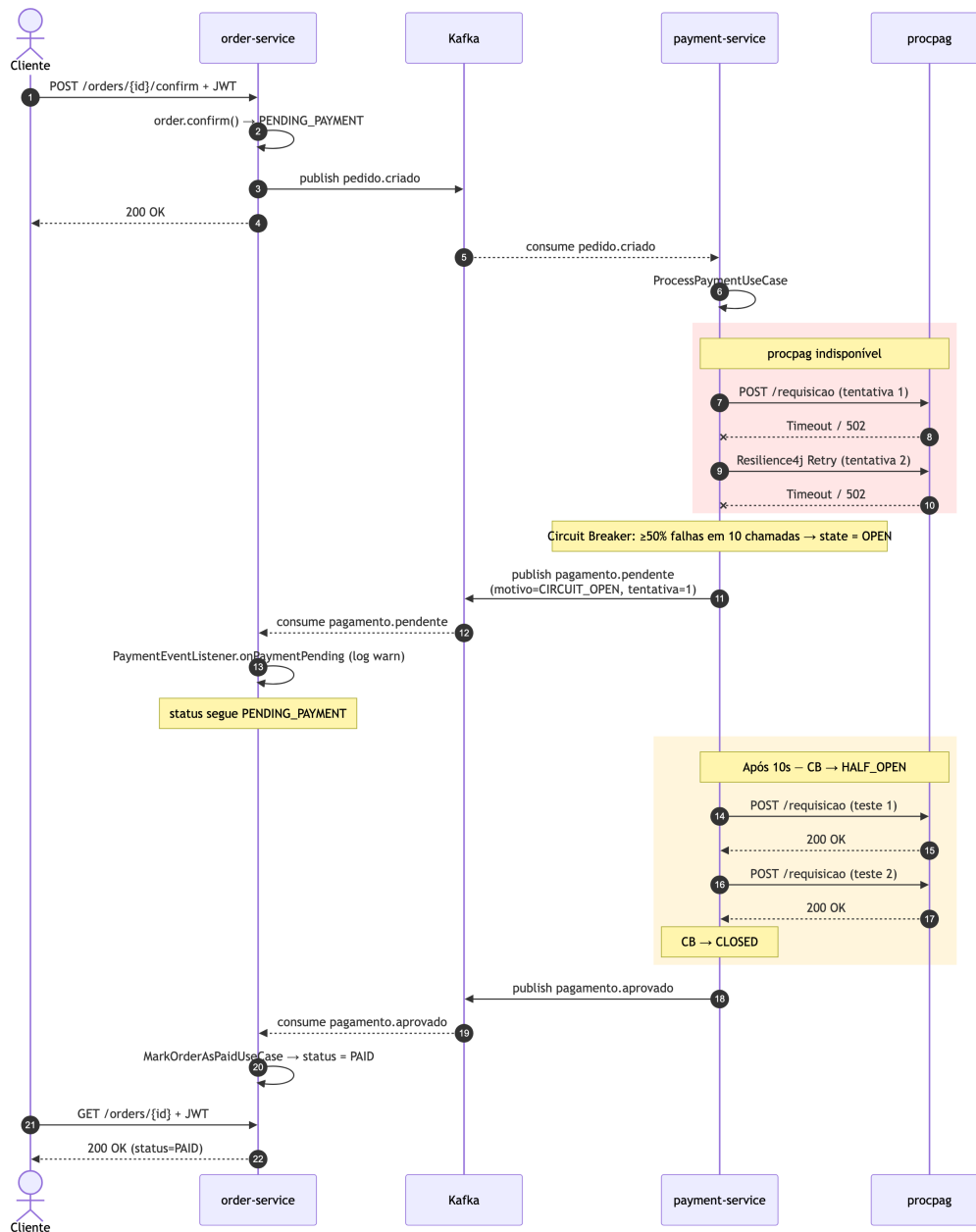
1.2 Diagrama de sequência

Demonstra o ciclo completo do pedido quando o procpag está disponível.



1.3 Diagrama de sequência — falha e recuperação

Demonstra o comportamento quando o procpag está indisponível e o Circuit Breaker entra em ação.



2. Descrição do fluxo principal de funcionamento

Etapa 1 — Autenticação

O cliente envia credenciais para `POST /api-users/v1/auth/login` no `auth-service`. Em caso de sucesso, recebe um JWT assinado em HS256 contendo os claims `sub` (email), `userId` (id numérico) e `userRole` (CLIENT ou ADMIN). Esse token deve ser apresentado no header `Authorization: Bearer <token>` em todas as chamadas subsequentes.

Etapa 2 — Criação do pedido

O cliente envia `POST /api-orders/v1/orders` com a lista de itens (`menuItem` + `quantity`). O `order-service` extrai o `clientId` do JWT, busca o nome e preço de cada item no catálogo local (`menu_items`) — desconsiderando qualquer valor enviado pelo cliente — e cria um `Order` em estado `AWAITING_CONFIRMATION`. Retorna 201 com o total calculado.

Etapa 3 — Confirmação e publicação do evento

O cliente envia `POST /api-orders/v1/orders/{id}/confirm`. O `order-service` valida a posse do pedido, transiciona o status para `PENDING_PAYMENT`, persiste e publica o evento `pedido.criado` no tópico Kafka homônimo, usando o `orderId` como chave de partição. Responde 200 imediatamente — o processamento do pagamento segue de forma assíncrona.

Etapa 4 — Processamento do pagamento

O `payment-service` consome `pedido.criado`, converte para o contrato esperado pelo `procpag` (`pagamento_id`, `cliente_id`, `valor`) e chama `POST /requisicao` no `procpag` através do `ProcpagHttpAdapter`, que aplica `Resilience4j`. Em caso de sucesso (HTTP 200), publica `pagamento.aprovado`; em caso de falha persistente, publica `pagamento.pendente`.

Etapa 5 — Atualização de status

O `order-service` consome os tópicos `pagamento.aprovado` e `pagamento.pendente` via `PaymentEventListener`. Ao receber `pagamento.aprovado`, o `MarkOrderAsPaidUseCase` (idempotente) transiciona o status do pedido para `PAID`. Ao receber `pagamento.pendente`, apenas registra o evento; o pedido permanece em `PENDING_PAYMENT` aguardando a próxima tentativa.

Etapa 6 — Consulta

O cliente pode consultar o pedido a qualquer momento via `GET /api-orders/v1/orders/{id}` ou listar todos os seus via `GET /api-orders/v1/orders`. Em ambos os casos o `order-service` valida que o pedido pertence ao `clientId` do JWT, retornando 403 caso contrário.

3. Pontos de resiliência

A resiliência do sistema é composta por duas camadas: padrões explícitos com `Resilience4j` na chamada ao serviço externo, e padrões arquiteturais distribuídos pelos serviços.

3.1 Resilience4j na chamada ao procpag (RNF-5.4)

Aplicado no payment-service, classe ProcpagHttpAdapter. Configuração em application.yml.

Padrão	Configuração	Comportamento
Timeout	2 segundos	A chamada é abortada se o procpag não responder em 2s
Retry	2 tentativas, espera de 300ms entre elas, apenas em IOException, ConnectException e ResourceAccessException	Tenta novamente em falhas transientes de rede
Circuit Breaker	Janela deslizante de 10 chamadas; abre quando $\geq 50\%$ falham; permanece OPEN por 10 segundos; HALF_OPEN permite 2 chamadas de teste	Bloqueia chamadas em massa quando o procpag está degradado, evitando esgotamento de recursos
Fallback	PaymentProcessingException lançada e capturada pelo error handler do Spring Kafka	Após retries esgotados ou CB aberto, publica pagamento.pendente em vez de propagar erro para o cliente

3.2 Comunicação assíncrona via Kafka

Toda a integração entre order-service e payment-service é mediada por Kafka, eliminando a necessidade de chamadas síncronas entre eles. Consequência: indisponibilidade de qualquer dos dois não causa erro no cliente final, pois as mensagens permanecem no tópico até serem consumidas.

3.3 Idempotência no consumo de eventos

O MarkOrderAsPaidUseCase valida o estado atual do pedido antes de aplicar a transição, garantindo que mensagens duplicadas (cenário esperado em Kafka) não causem efeito colateral:

Estado atual	Comportamento
Pedido não encontrado	Loga aviso e ignora
Já está PAID	Loga e ignora
PENDING_PAYMENT	Aplica a transição e persiste
Outro estado (ex.: CANCELLED)	Loga aviso e ignora

3.4 Dead Letter Topic (DLT)

O `payment-service` está configurado com `DeadLetterPublishingRecoverer` no `error handler` do Spring Kafka. Mensagens que falham repetidamente são enviadas ao tópico `pedido.criado-dlt`, evitando bloqueio do consumer e permitindo análise posterior.

3.5 Healthchecks no Docker Compose

Os bancos MySQL e o Kafka possuem `healthcheck` definido no `compose.yml`. Os serviços de aplicação declaram dependência `condition: service_healthy`, garantindo que apenas iniciam quando suas dependências estão prontas para receber conexões.

3.6 Isolamento de bancos por serviço

`auth-service` e `order-service` têm bancos MySQL independentes. Uma falha em qualquer um dos bancos afeta apenas o serviço dono — não derruba o sistema todo.

3.7 Tolerância a falhas no filtro JWT

O `JwtAuthenticationFilter` do `order-service` captura `JwtException` durante a validação. Tokens inválidos, malformados ou expirados não causam erro 500: o contexto de segurança simplesmente não é populado e o Spring Security responde 401/403 conforme o endpoint exigir autenticação.

3.8 Snapshot de preço no item do pedido

No momento da criação do pedido, o `order-service` copia `name` e `unitPrice` do `MenuItem` para dentro do `OrderItem`. Isso garante que alterações futuras de preço no catálogo **não** alterem retroativamente o valor de pedidos já criados — uma forma de resiliência a mudanças de dados.
